Rapport technique — POC RAG Puls-Events

Assistant de recommandation d'événements culturels · Périmètre Paris · 12/09/2026

1. Contexte et objectif

Puls-Events exploite une plateforme de découverte d'événements culturels en temps réel, alimentée notamment par OpenAgenda. L'entreprise souhaite tester un chatbot capable de fournir des recommandations personnalisées, augmentées par des réponses précises tirées des données d'événements. Ce document décrit la preuve de concept (POC) d'un système RAG (Retrieval-Augmented Generation / génération de réponses augmentée par la recherche).

Le POC devait démontrer la faisabilité technique avant une éventuelle industrialisation. Le périmètre géographique retenu est Paris. Seuls les événements de moins d'un an (historique de 12 mois et dates à venir) sont conservés, conformément à la demande du responsable technique.

2. Problématique

Un LLM seul connaît mal la programmation culturelle récente : il hallucine des dates, des salles ou des tarifs. À l'inverse, une recherche mot-clé sur un catalogue d'événements rate les formulations naturelles (« une expo calme ce week-end », « un concert jazz pas trop cher »). Le RAG combine les deux : recherche sémantique dans une base vectorielle, puis rédaction contrainte par les extraits trouvés.

- Source de vérité : API publique OpenAgenda (événements Parisiens).
- Index local FAISS, reconstructible à la demande.
- Génération et embeddings via la plateforme Mistral, orchestrés par LangChain.
- Évaluation sur un jeu de questions/réponses annoté (même information / même sens).

3. Architecture du système

Le système a deux phases. La phase d'indexation (hors ligne) récupère, filtre, découpe et vectorise les événements. La phase de question (en ligne) embarque la requête, interroge FAISS, construit un prompt augmenté et appelle le LLM Mistral. Aucun historique de conversation n'est utilisé dans le POC, comme demandé : chaque question est indépendante.

Chaîne d'indexation : OpenAgenda → JSON brut → filtres Paris + fraîcheur → documents LangChain → chunks → mistral-embed → FAISS.

Chaîne de question : texte utilisateur → embedding → top-k extraits FAISS → prompt système Puls-Events → mistral-small-latest → réponse + sources affichées.

Les métadonnées (titre, lieu, code postal, dates, URL) voyagent avec chaque chunk. Elles servent à la fois à l'affichage des sources dans la démo et aux tests unitaires de conformité du périmètre.

4. Collecte et préparation des données

La collecte s'appuie sur l'API REST OpenAgenda v2, authentifiée par clé publique de lecture. Le script tente d'abord une recherche transverse d'événements (filtre ville = Paris, événements en cours et à venir), puis complète via la découverte d'agendas officiels dont le titre ou le slug contient « Paris » (dont les slugs de type que-faire-a-paris).

Chaque événement est normalisé : textes localisés (fr), HTML retiré, timings convertis en dates ISO, lieu, adresse, code postal, coordonnées GPS. Un événement est retenu s'il est localisé à Paris (nom de ville, code postal 75xxx, ou point GPS dans la boîte englobante intramuros) ET si sa dernière date est postérieure à aujourd'hui moins 365 jours. Les événements trop pauvres textuellement sont écartés.

Dans l'état actuel du POC, 200 événements structurés sont disponibles pour l'indexation, découpés en 200 chunks. Exemples de lieux indexés : 139 Rue du Temple, 75003 Paris, 59 Rivoli, AUBERGE DE JEUNESSE MIJE MARAIS FOURCY, Académie du Climat, Ambassade de Lituanie, Artazart, Atelier Arnaud Marie, Atelier Néerlandais. Thématiques observées : 1970, 33ème Quinzaine du cinéma francophone, ALAN LUCIEN ØYEN ANTIGONE Avec 9 danseurs - CRÉATION - NORVÈGE, Adultes, Ambassade de Norvège, Antal Szerb, Archipel Chaos Monde, Art contemporain, Artisanat, Atelier.

Le découpage (chunking) utilise RecursiveCharacterTextSplitter de LangChain (1 200 caractères, chevauchement 150). Un événement court reste un seul chunk ; une longue description est fragmentée tout en conservant titre, lieu et dates dans les métadonnées de chaque fragment, pour ne pas perdre le contexte lors de la recherche.

5. Choix techniques

LangChain a été retenu parce que le brief l'impose et parce qu'il orchestre proprement retriever, prompt et LLM (LCEL). FAISS CPU (IndexFlatIP, similarité cosinus via vecteurs L2-normalisés côté LangChain/FAISS) convient au volume d'un POC : quelques centaines de vecteurs, recherche exacte, zéro infrastructure. Un index IVF ou HNSW n'apporterait rien à cette échelle et compliquerait la reconstruction.

Mistral est utilisé de bout en bout : mistral-embed (dimension 1024) pour la vectorisation, mistral-small-latest pour la génération. Ce couplage évite un décalage de « langue vectorielle » entre l'index et le modèle de chat. mistral-small-latest a été préféré à mistral-large pour le POC : coût et latence plus bas, français de bonne qualité, suffisant dès lors que le contexte RAG est pertinent. Un passage à un modèle plus grand reste un levier de la version finale.

L'application de démo est un Streamlit local. Les secrets restent dans un fichier .env ignoré par Git. La reconstruction de l'index est un script unique (rebuild_index.py), reproductible dans un environnement virtuel documenté.

6. Modèles sélectionnés

- Embeddings : mistral-embed — texte d'événement → vecteur, recherche sémantique.
- Génération : mistral-small-latest — rédaction des recommandations à partir des extraits.
- Température 0,2 — réponses plutôt factuelles, peu créatives.
- k = 5 extraits — compromis entre couverture et bruit dans le prompt.

Le prompt système impose le français, l'interdiction d'inventer des événements absents du contexte, et la restitution du titre, du lieu, des dates et de l'URL. Si le contexte est insuffisant, le modèle doit le dire. C'est le principal garde-fou anti-hallucination du POC.

7. Tests et évaluation

Les tests unitaires (tests/test_events_freshness.py) vérifient les filtres sur des événements synthétiques (Paris vs Lyon, date récente vs date trop ancienne) puis, lorsque les fichiers existent, parcourent le JSON traité et les métadonnées FAISS pour garantir que tout document indexé est bien à Paris et dans la fenêtre d'un an.

Un jeu annoté de questions/réponses est généré à partir des événements réels (vérité terrain : titre, lieu, période). Selon le brief, une réponse du système est correcte si elle a le même sens et contient les mêmes informations que l'annotation humaine. L'évaluateur mesure la couverture des faits obligatoires (titre, lieu) dans la réponse générée.

Résultats du POC : 12 questions annotées, accuracy retrieval = 0.667, couverture moyenne des faits dans les extraits = 0.667. Le critère du brief (même sens / mêmes informations) est d'abord vérifié au niveau du retrieval : si le bon événement n'est pas dans le top-k, le LLM ne peut pas répondre correctement. L'évaluation des réponses générées (`python evaluate.py --llm`) dépend du quota de l'API Mistral (le palier gratuit peut renvoyer HTTP 429). Dans ce cas, la démo live reste valable dès que le quota se libère.

8. Résultats qualitatifs du POC

Le système répond de façon exploitable sur des questions du type « parle-moi de [titre] », « que se passe-t-il à [salle] », ou « une sortie dans le 75011 ». Les sources s'affichent sous la réponse, ce qui permet à un product owner de vérifier l'origine OpenAgenda. La reconstruction de l'index tient en une commande. L'environnement se installe via requirements.txt.

Limites assumées du POC : pas de mémoire de dialogue, pas de filtre date interactif dans l'UI (la fraîcheur est garantie à l'indexation), volume plafonné (MAX_EVENTS) pour rester dans le quota gratuit Mistral, dépendance réseau aux deux API, et similarité sémantique brute sans reranker.

9. Recommandations pour la version finale

- Filtres métadonnées au moment de la recherche : arrondissement, intervalle de dates, gratuit/payant, type d'événement.
- Recherche hybride (mots-clés BM25 + vecteurs) pour les titres propres et les noms de salles.
- Reranker (modèle croisé) sur les 20 premiers extraits avant le LLM.
- Quota et cache d'embeddings : ne re-vectoriser que les événements nouveaux ou modifiés.
- Observabilité : journaliser question, extraits, réponse, et un score de fidélité au contexte.
- Jeu annoté enrichi manuellement par l'équipe produit (20–50 questions représentatives des vrais utilisateurs).
- Passer à un modèle Mistral plus capable si les réponses manquent de synthèse, tout en gardant une température basse.
- Déploiement : API FastAPI derrière le front Puls-Events, index FAISS ou bascule vers un moteur managé si le catalogue dépasse ~100k chunks.
- Conformité : licences OpenAgenda, mentions des sources, pas de stockage inutile de données personnelles.

10. Conclusion

Le POC démontre qu'un assistant de recommandation culturelle, ancré dans OpenAgenda et limité

à Paris sur une fenêtre d'un an, est réalisable avec la pile imposée LangChain + Mistral + FAISS, entièrement reproductible en local. La génération augmentée réduit les hallucinations par rapport à un LLM nu, à condition que le retrieval soit pertinent. Les prochains investissements doivent porter sur les filtres, le reranking et l'évaluation produit, plus que sur un changement d'infrastructure.

Annexes

- Commandes : python check_setup.py · python rebuild_index.py · pytest · streamlit run app.py · python evaluate.py
- Fichiers clés : src/pulsevents/ · tests/test_events_freshness.py · data/evaluation/annotated_qa.json
- Secrets : fichier .env (jamais versionné).